Scientific Computing WS25/26

Hande Pamuksuz

Topics to be covered

1 – Structs

2 – Rustc Compiler

3- Rust Styling

What is a struct?

- User-defined data type in Rust
- Commonly used to group together related data
- Similar to tuples
  - Each item in tuple/struct can be of different type
- Different from tuples
  - User can individually name each item
  - Name of Struct: User
  - Field names:
    - active, username, email, sign-in
  - Field types:
    - Bool, String, String, u64

By using a struct, we are able to maintain related data and names of data => CLARITY

```
struct User {
    active: bool,
    username: String,
    email: String,
    sign_in_count: u64,

}
```

How to use a Struct?

- To use a struct, we need an INSTANCE

```
fn main() {
    let mut user1 = User {
        active: true,
        username:String::from("someusername123"),
        email: String::from("someone@example.com"),
        sign_in_count: 1,
    };

    user1.email = String::from("anotheremail@example.com");
}
```

- DOT notation: to access fields of instance
- To change field values (user1.email above) the instance has to be **MUTABLE**.

Builder function

- To create multiple instances easier in the code, a builder function can be created which will create the struct within it.

```
fn build_user(email: String,
username: String) -> User {
    User {
        active: true,
        username: username,
        email: email,
        sign_in_count: 1,
    }
}
```

Field init shorthand

- When initializing the instance, if the key input has the same name as the field, the repetition can be avoided.

```
fn build_user(email: String, username:
String) -> User {
    User {
        active: true,
        username,
        email,
        sign_in_count: 1,
    } }   }
```

Update Syntax

- Using an existing instance, the struct can be used to create another similar instance.

```
fn main() { // --snip-
    let user2 = User {
        active: user1.active,
        username: user1.username,
        email: String::from("another@example.com"),
        sign_in_count: user1.sign_in_count, };
}
```

```
fn main() { // --snip-
    let user2 = User {
        email: String::from("another@example.com"),
        ..user1 }; }
```

Complex data like String gets moved. You lose it from the original. (no more user1.email, it’s gone)

Simple data like integers and booleans (sign in, active) gets copied. The original keeps its copy. If I assign a new username and email to user2, but keep the sign in and active values from user1, then USER1 IS STILL ACCESSIBLE. This is because we didn't move the complex data, only simple data.

```
let user2 = User {
    email:
String::from("another@example.com"),
  ..user1
};
```

Struct Tuple:

```
struct Color(i32, i32, i32);
struct Point(i32, i32, i32);

fn main() {
    let black = Color(0, 0, 0);
    let origin = Point(0, 0, 0);
}
```

Unit-Like Tuple (without any fields):

```
struct AlwaysEqual;

fn main() {
    let subject = AlwaysEqual;
}
```

It’s possible for structs to store references to data owned by something else, but to do so requires the use of *lifetimes.*

Much cleaner way to list functions for a struct.
To use -> Syntax: instance.method().

impl

```
struct Rectangle {
  width: u32,
  height: u32, }

impl Rectangle {
  fn area(&self) -> u32 {
    self.width * self.height }
}

fn main() {
  let rect1 = Rectangle { width: 30, height: 50, };
  println!( "The area of the rectangle is {} square
  pixels.", rect1.area() ); }
```

Methods

```
impl Rectangle {
  fn area(&self) -> u32 {
    self.width * self.height }
}

  fn can_hold(&self, other:&Rectangle) -> bool {
    self.width > other.width && self.height > other height }
}
```

Example of a method with more than one parameter.

Functions under the same impl block are “associated” functions.

Topics to be covered

1 – Structs

2 – Rust Style Guide

3- Rustc

The ideal code is:

- Readable by all -> suitable for teams, multiple users.
  - Easy to understand and edit.
- Consistent
- Reduces mental effort

`cargo fmt` automatically rewrites your Rust code to follow the official style guide, making it consistently formatted and more readable.

- "Recommended"

- **Rustfmt**
  - Component to use cargo fmt
  - To use:

```
$ rustup component add rustfmt
$ rustfmt main.rs
$ cargo fmt
```

Rust fmt, alternative uses

```
# See what would change without actually changing files
cargo fmt -- --check

# Or see the differences
cargo fmt -- --verbose

# Format a specific file
cargo fmt -- src/main.rs

# Format multiple specific files
cargo fmt -- src/main.rs src/lib.rs

# Format and then display differences
cargo fmt && git diff
```

Key Formatting Rules

Indentation: 4 spaces (no tabs)

Line length: Max 100 characters per line

Bracing on the same line: {}

Closing brace is not indented, on next line.

Spaces around operators: x + y not x+y

- Space after ":" , space before and after "=".
- No space before ";"
- No space after "!", ex. !x

- Prefer line comments //

In cargo.toml: [packages] first**, [dependencies]** later

Naming:

- **UpperCamelCase** for types (struct MyStruct)
- **snake_case** for all variables/functions (my_function)
- **SCREAMING_SNAKE_CASE** for constants (MAX_SIZE)

```
fn build_user(email: String,
username: String) -> User {
    User {
        active: true,
        sign_in_count: 1,
    }
}
```

Topics to be covered

1 – Structs

2 – Rust Style Guide

3- Rustc

- **`rustc`** is a Rust compiler
- Most commonly used through **`cargo`**

- **`cargo:`** manages projects (multiple files)
- **`rustc:`** compiles single files
- Using cargo is more realistic in real-life projects

Why is rustc useful?

1. **Compiles** Rust code into **executable program**
2. **Checks** code for **errors** before running

This check leads to early catching of **bugs.**

- With **`rustc`**
  - Code will not compile if unsafe
  - Memory issues are avoided
  - Error messages are clear and exact, fixes are suggested (see help: ..)
    - (also possible: rustc --explain EXXXXX
  - Optimizes the code automatically
  - Memory safety without garbage collection
    - No manual memory management
    - No runtime overhead
    - Safe, fast, predictable

```
$ cargo run
   Compiling structs v0.1.0 (file:///projects/structs)
error[E0106]: missing lifetime specifier
 --> src/main.rs:3:15
  |
3 |     username: &str,
  |               ^ expected named lifetime parameter
  |
help: consider introducing a named lifetime parameter
  |
1 ~ struct User<'a> {
2 |     active: bool,
3 ~     username: &'a str,
  |

error[E0106]: missing lifetime specifier
 --> src/main.rs:4:12
  |
4 |     email: &str,
  |            ^ expected named lifetime parameter
  |
help: consider introducing a named lifetime parameter
  |
1 ~ struct User<'a> {
2 |     active: bool,
3 |     username: &str,
4 ~     email: &'a str,
  |
```

Summary

1 – Structs:

Structs bundle related data together and methods give them function, keeping your files organized and with clear relations.

2 – Rust Style Guide

`cargo fmt` to automatically formats code consistently, so the user can focus on their logic instead of formatting for easier legibility.

3- Rustc

rustc catches memory bugs and errors at compile time with helpful messages, making code files safe and fast.

Thank you for your attention.

References:

https://doc.rust-lang.org/stable/book/ch05-00-structs.html
https://doc.rust-lang.org/nightly/style-guide/index.html
https://doc.rust-lang.org/stable/rustc/what-is-rustc.html
(accessed 04.11.2025)